Il package `view` — architettura e divisione dei compiti

Progetto Mesos · Client-side rendering · package `it.polimi.ingsw.view` · ~11.000 righe, 45 classi

Il package `view` contiene **tutto e solo il livello di presentazione** del client. Non possiede logica di gioco né di rete: è uno strato passivo che *legge* lo stato dal `ClientModel` e *invia* le azioni dell'utente attraverso il `ClientController`. È organizzato in due interfacce grafiche complete e intercambiabili — una testuale (**TUI**) e una a finestre (**GUI** in JavaFX) — più un sottopackage di **dati di carta condivisi** che entrambe riusano. La regola di fondo è esplicitata nel contratto comune: *«cosa» succede è condiviso, solo «come» viene disegnato cambia.*

1. Il contratto comune e il modello "push"

Il cuore architetturale è l'interfaccia `View` (alla radice del package). Definisce l'insieme di reazioni che **ogni** interfaccia deve implementare: callback di output come `onLobbyChanged()`, `onGameStarted()`, `onGameStateChanged()`, `onBoardChanged()`, `onEventResolved()`, `onGameEnded()`, `onError()`, `onConnectionLost()`, oltre a `run()` e ai metodi della classifica. Sia `Tui` che `Gui` implementano questa interfaccia.

Il flusso segue un **modello push a singolo driver**: il `ClientController`, dopo aver applicato al `ClientModel` il messaggio ricevuto dal server, invoca il callback corrispondente sulla view. La view **non interroga mai attivamente la rete**: si limita a ridisegnare lo stato corrente quando viene notificata. È importante notare cosa *non* è nell'interfaccia: le **azioni** dell'utente (login, pesca carta, posiziona totem…) non sono dichiarate qui, perché sono metodi concreti già pronti sul `ClientController`, chiamati in modo identico dal parser di comandi della TUI e dai bottoni della GUI. Questo elimina la duplicazione della logica di interazione tra le due interfacce.

Codice realmente condiviso tra TUI e GUI: l'interfaccia `View`, il sottopackage `view.card` (descrizioni e punteggi delle carte) e `EndGameHelper` (catena ufficiale di spareggio: più PP → più cibo → vittoria condivisa), così che il vincitore sia calcolato in modo identico nelle due interfacce.

2. Struttura dei sottopackage

Sottopackage	File	Ruolo
`view (radice)`	2	`View` (contratto), `EndGameHelper` (spareggio vincitori condiviso)
`view.card`	7	Dati e testi delle carte, condivisi da TUI e GUI
`view.tui`	7	Interfaccia testuale completa (rendering ANSI da terminale)
`view.gui`	2	Bootstrap JavaFX e adattatore `View`
`view.gui.asset`	3	Caricamento e cache di immagini/asset grafici
`view.gui.component`	23	Componenti visuali della board JavaFX (la parte più grande)
`view.gui.util`	4	Helper trasversali GUI: animazioni, audio, icone, palette colori

3. Il sottopackage condiviso `view.card`

Prima di descrivere le due interfacce conviene isolare ciò che le accomuna. `view.card` trasforma i dati grezzi delle carte (file JSON) in informazioni pronte da mostrare, **senza dipendere né dalla TUI né dalla GUI**.

- `CardDataCatalog` — carica `characters.json`, `buildings.json`, `events.json` e li conserva restituendo sempre copie immutabili (così la view non può alterare il catalogo).
- `CharacterType`, `BuildingEffect`, `EventType` — modellano i tre tipi di carta tramite **polimorfismo**: ogni tipo conosce le proprie righe di testo descrittive (`ruleLines`), la propria icona e il proprio metodo di punteggio, evitando catene di `switch` sul tipo nei chiamanti.
- `CardJson` — lettura difensiva dei valori JSON con default, per non ripetere i controlli ovunque.
- `CharacterTally` e `BuildingScoring` — strutture di appoggio che ogni tipo usa per registrare il proprio contributo al riepilogo della tribù e per calcolare i punti di fine partita.

4. Il sottosistema TUI (`view.tui`)

La TUI è un'interfaccia da terminale completa basata su **codici ANSI** e su un rendering a larghezza fissa. La divisione interna separa nettamente *chi disegna*, *chi calcola il testo* e *chi interpreta l'input*.

Classe	Righe	Responsabilità
`Tui`	630	Implementa `View`: thread di input, ciclo di login, gestione prompt e attese; ogni callback converge su un unico `refresh()`.
`Renderer`	1671	Compositore principale: assembla a schermo lobby, board, dock giocatori, eventi e classifiche.
`CardRenderer`	512	Disegna la singola carta come blocco `String[]` di dimensione fissa. Classe *pura*: non stampa e non legge nulla.
`CardCatalog`	272	Visitor sulle famiglie di carte per produrne la rappresentazione testuale.
`CommandParser`	148	Interpreta una riga di comando e la inoltra al `ClientController` (es. `pick`, `build`, `place`, `ready`).
`TuiStyle`	62	Sole costanti: codici ANSI e dimensioni fisse di pannelli e carte. Nessun comportamento.
`TuiText`	297	Helper puri di larghezza/allineamento calcolati sui caratteri *visibili* (ignorando le sequenze di escape).

Il principio guida è la **purezza dei renderer**: `CardRenderer` e `TuiText` non hanno effetti collaterali (non stampano, non leggono input), quindi sono facilmente testabili; solo `Tui` e `Renderer` toccano lo standard output. L'input gira su un thread dedicato e si sincronizza con i refresh tramite un lock sul modello.

5. Il sottosistema GUI (`view.gui` + `component`, `asset`, `util`)

La GUI è scritta in **JavaFX** e rappresenta la parte più ampia del package (oltre 30 classi). La sua divisione interna è a tre livelli: un *bootstrap*, un insieme di *componenti* autonomi della scena e un livello di *servizi trasversali*.

5.1 Bootstrap e adattatore (`view.gui`)

- `Gui` — implementa `View`: è il sottile adattatore che riceve i callback dal controller e li marshalla sul thread JavaFX, delegando il disegno ai componenti.
- `GuiApplication` (≈830 righe) — la `Application` JavaFX: gestisce lo `Stage`, il tema, il cambio di schermata (login → lobby → board → fine partita), gli overlay globali (regole, sospensione) e i toast.

5.2 Componenti della scena (`view.gui.component`)

Ogni componente è una classe `final` a responsabilità singola, che costruisce e aggiorna una porzione dell'interfaccia leggendo dal `ClientModel`. Le schermate principali sono pubbliche (`LoginView`, `LobbyView`, `GameBoardView`, `EndGameView`, `LeaderboardView`), mentre i mattoni interni della board sono package-private.

Componente	Responsabilità
`GameBoardView`	Aggregatore della board: orchestra tutti i sotto-componenti seguenti.
`CardMarketView` / `RowController`	Le due fasce di mercato; `RowController` fa il *diff* col modello per ridisegnare solo il necessario e anima la distribuzione delle carte.
`OfferTilesView` / `TurnOrderView`	Tessere offerta e ordine di turno.
`PlayerDockView` / `TribeSummaryService`	Dock dei giocatori e riepilogo sintetico della tribù.
`DeckView`, `CardFocusView`, `CardDescriptionService`	Mazzi, pannello di carta ingrandita e testo-regole (separato dal layout).
`OverlayController` / `EventResolutionOverlay`	Overlay modali: il primo è il telaio generico, il secondo mostra l'evento d'era con i delta per giocatore.
`CardImageFactory` / `TotemMarkerFactory`	Factory di immagini-carta e di marcatori-totem.
`BoardScaler` / `TotemMoveAnimator` / `MesosLogo`	Scalatura uniforme della board, animazione di spostamento totem, logo.
`GuiViews`	Mini-helper statici condivisi tra i componenti (spaziatori, titoli, nome turno corrente…).

5.3 Asset e utilità (`view.gui.asset`, `view.gui.util`)

- `GuiAssetLoader`, `GuiAssetCatalog`, `CardAsset` — caricano e **memorizzano in cache** le immagini (le `Image` JavaFX sono immutabili e condivisibili tra più `ImageView`).
- `GuiAnimations`, `GuiAudio`, `GuiIcons`, `TotemPalette` — servizi trasversali: transizioni riutilizzabili, effetti sonori, icone e l'unica fonte di verità per il colore di ciascun totem.

6. Sintesi della divisione dei compiti

La gerarchia di responsabilità del package è chiara e a livelli ben separati:

- **Contratto** (`View`): definisce *quali* eventi la presentazione deve gestire, uguali per tutti.
- **Dati condivisi** (`view.card`, `EndGameHelper`): regole testuali, punteggi e spareggi, scritti una volta sola.
- **TUI**: renderer puri (testabili) + un solo punto che stampa + un parser di comandi.
- **GUI**: bootstrap JavaFX + componenti a responsabilità singola + servizi trasversali (asset, animazioni, audio).

> Il vantaggio principale di questo disegno è che le due interfacce restano **perfettamente intercambiabili e coerenti**: condividono modello, controller, regole delle carte e logica di vittoria, e si limitano a tradurre lo stesso stato in due forme di rappresentazione diverse — testo a terminale oppure scena JavaFX.